Python Arrays

An **array** stores many values under one name; list, `array.array` and NumPy differ.

1) A list can mix types

Example :

```
marks = [80, 85, 90, 75, 95]
print(marks, type(marks))
print("First mark:", marks[0])
print("Last mark:", marks[-1])
mixed_data = [10, "Rahul", 85.5, True]
print(mixed_data)
```

Output :

```
[80, 85, 90, 75, 95] <class 'list'>
First mark: 80
Last mark: 95
[10, 'Rahul', 85.5, True]
```

2) array.array: a typed sequence

Example :

```
import array
numbers = array.array('i', [10, 20, 30])
numbers.append(40)
numbers.insert(1, 15)
numbers[0] = 5
print(numbers)
print("Number of elements:", len(numbers))
```

Output :

```
array('i', [5, 15, 20, 30, 40])
Number of elements: 5
```

the type code decides what fits: i means signed integer, d means double decimal

A decimal array

```
import array
temperatures = array.array('d', [28.5, 29.0, 30.2])
for temperature in temperatures:
    print(temperature)
```

Output :

```
28.5
29.0
30.2
```

Wrong data type

```
import array
integer_values = array.array('i', [10, 20, 30])
integer_values.append("Rahul")
```

Output :

```
TypeError: 'str' object cannot be interpreted as an integer
```

NUMPY ARRAYS

NumPy (Numerical Python) gives fast element-wise maths and multidimensional arrays.

1) Create, then element-wise maths

Example :

```
import numpy as np
numbers = np.array([10, 20, 30, 40])
print(numbers, numbers.dtype, numbers.ndim, numbers.shape)
print(numbers * 2)
maths_marks = np.array([70, 80, 90])
bonus_marks = np.array([5, 5, 5])
print(maths_marks + bonus_marks)
```

Output :

```
[10 20 30 40] int64 1 (4,)
[20 40 60 80]
[75 85 95]
```

*numbers * 2 multiplies each element, no loop needed, unlike array.array*

2) Two dimensions and statistics

Rows and columns

```
import numpy as np
marks = np.array([[85, 90, 80], [90, 85, 95]])
print(marks)
print(marks.shape)
print(marks[0])
print(marks[0, 0])
```

Output :

```
[[85 90 80]
 [90 85 95]]
(2, 3)
[85 90 80]
85
```

Statistics

```
import numpy as np
marks = np.array([85, 90, 75, 80, 95])
print(np.mean(marks))
print(np.max(marks), np.min(marks))
print(np.sum(marks))
print(round(np.std(marks), 2))
```

Output :

```
85.0
95 75
425
7.07
```

3) reshape

Example :

```
import numpy as np
numbers = np.array([10, 20, 30, 40, 50, 60])
matrix = numbers.reshape(2, 3)
print(matrix)
print(matrix.shape)
```

Output :

```
[[10 20 30]
 [40 50 60]]
(2, 3)
```

the element count must stay the same: 2 rows x 3 columns is 6 values

Async & Await

Async lets a program use waiting time (network, database) to make progress on other tasks.

1) Synchronous: one after another

Example :

```
import time
def download_file(file_name):
    print("Starting:", file_name)
    time.sleep(1)
    print("Finished:", file_name)
start = time.time()
for name in ["file_1", "file_2", "file_3"]:
    download_file(name)
print("Total time:", round(time.time() - start), "sec")
```

Output :

```
Starting: file_1
Finished: file_1
Starting: file_2
Finished: file_2
Starting: file_3
Finished: file_3
Total time: 3 sec
```

2) Asynchronous: overlapping the waits

Example :

```
import asyncio, time
async def download_file(file_name):
    print("Starting:", file_name)
    await asyncio.sleep(1)
    print("Finished:", file_name)
async def main():
    start = time.time()
    files = [download_file(n) for n in ("file_1", "file_2", "file_3")]
    await asyncio.gather(*files)
    print("Total time:", round(time.time() - start), "sec")
asyncio.run(main())
```

Output :

```
Starting: file_1
Starting: file_2
Starting: file_3
Finished: file_1
Finished: file_2
Finished: file_3
Total time: 1 sec
```

the waits overlap, so three 1 second waits take about 1 second in total

Keyword	Meaning
`async def`	defines a coroutine function
`await`	pause until done, letting other tasks run
`asyncio.gather()`	run several awaitables concurrently

Remember

`async` is not a magic speed switch: CPU-heavy code in an `async def` still blocks the event loop.

Async API & Practice

A FastAPI endpoint can be `async`, but a normal model prediction still needs a worker thread.

1) Breast Cancer API: the key lines

app.py (excerpt)

```
@app.post("/predict")
async def predict(input_data: PredictionInput):
    prediction = await run_in_threadpool(
        model.predict,
        [input_data.features]
    )
    label = int(prediction[0])
```

Output :

```
(runs inside the FastAPI server: uvicorn app:app --reload)
```

model.predict is normal synchronous work, so never write await model.predict directly

2) Practice

Task 1 Integer array: append, update.

Solution :

```
import array
values = array.array('i', [5, 10, 15, 20])
values.append(25)
values[0] = 100
print(values)
```

```
array('i', [100, 10, 15, 20, 25])
```

Task 2 NumPy: multiply by 3.

Solution :

```
import numpy as np
values = np.array([2, 4, 6, 8])
print(values * 3)
```

```
[ 6 12 18 24]
```

Task 3 NumPy: mean, max, min.

Solution :

```
import numpy as np
scores = np.array([72, 85, 90, 68, 95])
print("Mean:", np.mean(scores))
print("Maximum:", np.max(scores))
print("Minimum:", np.min(scores))
```

```
Mean: 82.0
Maximum: 95
Minimum: 68
```

Task 4 Three orders concurrently.

Solution :

```
import asyncio
async def prepare_order(order_id):
    print("Preparing order:", order_id)
    await asyncio.sleep(1)
    print("Completed order:", order_id)
async def main():
    orders = [prepare_order(i) for i in (101, 102, 103)]
    await asyncio.gather(*orders)
asyncio.run(main())
```

```
Preparing order: 101
Preparing order: 102
Preparing order: 103
...
```

List vs array.array vs NumPy

Feature	list	array.array	NumPy array
Mixed types	yes	no (type code)	one common dtype
Element-wise maths	loops	no	yes
Dimensions	nested lists	one	many